



UNIVERSITY OF
BIRMINGHAM

Computational Authorship Analysis Part II

Authorship Attribution via Large Language Models

Weihang Huang

Department of Linguistics and Communication

University of Birmingham

Table of Contents

- Session 1(14:00-15:30)
 - A: Introduction to Large Language Model
 - Definition
 - Pipeline
 - Types
 - B: Introduction to Authorship Attribution via Language Models
 - Authorial Language Models(ALMs)
- Session 2(15:45-17:00)
 - Practicum on Authorship Attribution via Language Models

Session 1A

Introduction to Large Language Model

What is an LLM?

- A text generator?
- A bot that can answer your question?
- ...
- Or a model that can predict the probability of tokens.

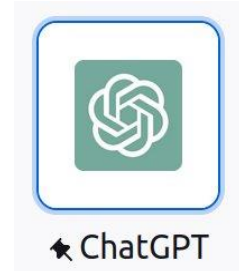
Open GLM

ANTHROPIC

Talk to
Claude



Llama 2



OpenChatKit

Large Language Model(LLM)

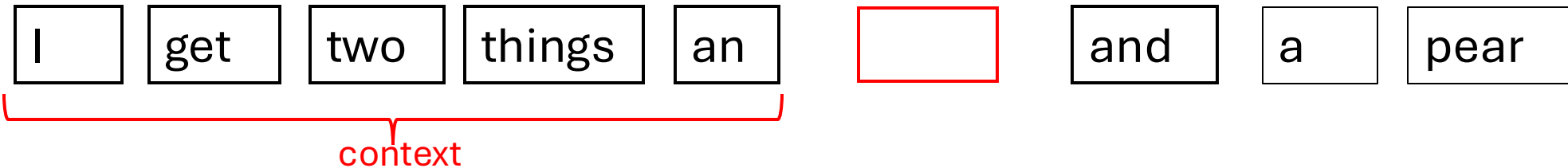
- **LLM** predicts probability of tokens based on their context.
- Input: Tokenized Text
- Output: Probabilities of token(s)

I get two things an

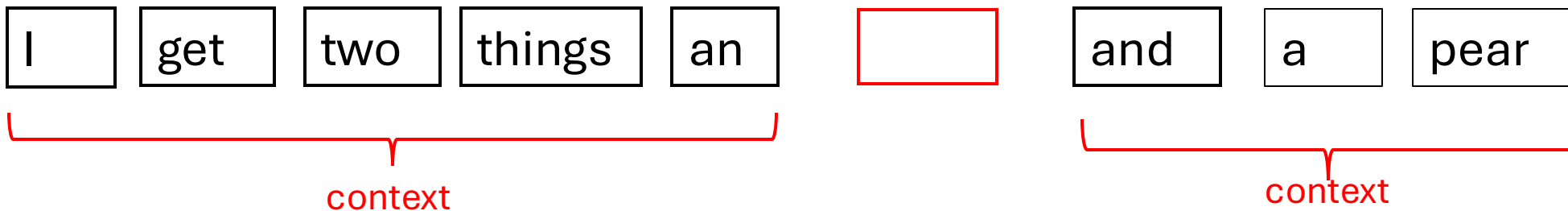
Token	Prob
apple	0.63
orange	0.32
pear	0.01
...	...

Types of Large Language Models

- Causal Language Model(CLM) predict probability based on previous tokens only.
 - (useful in text generation, predictability check)



- Masked Language Model(MLM) predict probability based on both previous and next tokens.
 - (useful in natural language understanding tasks)



LLM versus ChatBot

- Large Language Model's prediction is based on context only.
 - Text generated is a "continuation of writing"
- Chatbot's prediction is based on not only context, but also the LM policy that align model to users' need.
 - Text generated is the result that follows instructions from users.

You are a pirate and you are going to introduce the history of the printing press in 300 words.



LLM
Text
Generation

It was a chilly winter evening in 1632 when the unthinkable happened. The streets of Strasbourg, Germany, were abuzz with the sound of drums and the smell of gunpowder wafted through the air. A pirate ship had sailed up the Rhine River, its black sails billowing in the wind.



ChatBot

Yer lookin' fer the tale o' how the printin' press came to be, eh? Well, settle yerself down with a pint o' grog and listen close.

It were Johannes Gutenberg, a German blacksmith's apprentice, who set sail fer the world o' printin' in the 15th century. In 1440, he invented the movable-type press, which allowed for mass production o' books.

Large Language Model(LLM)

- **LLM** predicts probability of tokens based on their context.
- Input: Tokenized Text
- Output: Probabilities of token(s)

I get two things an

Token	Prob
apple	0.63
orange	0.32
pear	0.01
...	...

Input: Tokenized Text

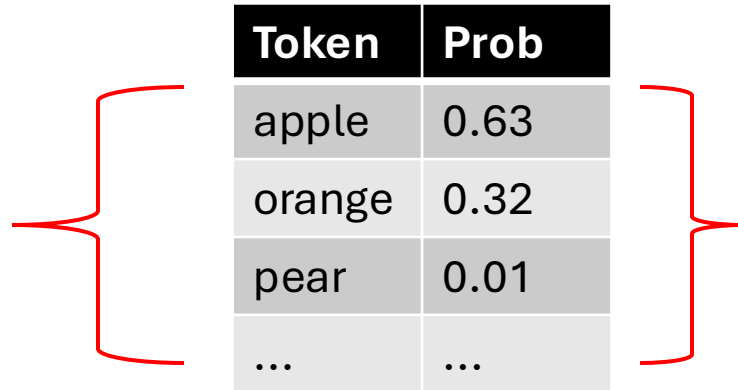
- Text needed to be tokenized before inputted into LLM
- Tokenizer is a module that responsible of preparing the inputs for an LLM
 - Input: String
 - Output: List of tokens(e.g. words), typically as index of dictionary
- Typical Tokenization Algorithm for LLM: Byte Pair Encoding(BPE)

Example: Tokenization with BPE

- Cookiescookcookiesincooker
- A=cook
- AieAAiesinAer
- B=ies
- ABAABinAer
- Recursive! C=AB(cookies)
- CACinAer
- Lexicons= [A B C in er], which is [cook, ies, cookies, in ,er]
- Tokenized text(numerical)= [2,0,2,3,4]

Output: Probability of Token(s)

LLM producible lexicon
(defined by tokenizer)



Token	Prob
apple	0.63
orange	0.32
pear	0.01
...	...

Probability

Text Generation

- Probability of tokens can be used for **text generation**.

I get two things an apple and a pear

Token	Token	Token	Token	Prob
apple	and	a	pear	0.56
orange	,a	an	orange	0.3
pear	pear	.	apple	0.01
...	...	...	...	...

Language Modeling for Text

- Probability of tokens can be calculated recursively from the first token to the last token of a text of interest.

The diagram shows the sentence "I get two things an apple and a pear" at the top. Below it are eight tables, each representing the probability distribution over words given the prefix up to the previous word. Red arrows connect each word in the sentence to its corresponding table.

Token	Prob
I	0.63
you	0.32
him	0.01
...	...

Token	Prob
get	0.65
have	0.2
take	0.02
...	...

Token	Prob
two	0.55
one	0.4
three	0.02
...	...

Token	Prob
things	0.5
stuff	0.32
apples	0.01
...	...

Token	Prob
an	0.55
a	0.4
.	0.01
...	...

Token	Prob
apple	0.63
orange	0.32
pear	0.01
...	...

Token	Prob
and	0.65
,a	0.2
pear	0.02
...	...

Token	Prob
a	0.55
an	0.4
.	0.02
...	...

Token	Prob
pear	0.56
orange	0.3
apple	0.01
...	...

LLM Output: Probability Matrix

LLM Producing Lexicons

Input
Tokens

	...	a	abandon	...	an	...	apple	...
I	...	0.02581	0.07568	...	0.06320	...	0.03982	...
get	...	0.00039	0.00399	...	0.09626	...	0.04648	...
two	...	0.00185	0.00996	...	0.01128	...	0.24887	...
things	...	0.00390	0.00625	...	0.83658	...	0.00310	...
an	...	0.24771	0.01989	...	0.42341	...	0.21486	...
apple	...	0.00042	0.00016	...	0.08132	...	0.38014	...
and	...	0.00451	0.00288	...	0.16216	...	0.01336	...
a	...	0.23962	0.00017	...	0.11122	...	0.13319	...
pear	...	0.00025	0.00042	...	0.00285	...	0.12497	...

Probability of the Ground Truth Token

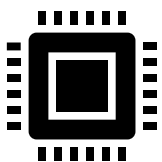
*Ground truth token are the token that are already known in text.

[illegible]

Predictability in Probability of Ground Truth Token

- Probability of ground truth token is a measure of predictability
 - Ground truth token always have probability of 1 in corpus(because it occurs there)
 - The further the predicted probability is deviated from 1, the worse the prediction
 - OR
 - The smaller the predicted probability is, the worse the prediction(because max prob =1)

I get two things an apple



Token	Prob
apple	0.63
orange	0.32
pear	0.01
...	...



Token	Prob
apple	1
orange	0
pear	0
...	0

Predictability in Probability of Ground Truth Token

- Predictability of multiple tokens(e.g. text) can be accessed via pooling of these probability via multiplication

I get two things an apple

$p(\text{an} | \text{I get two things}) = 0.4$ (A)

$p(\text{apple} | \text{I get two things an}) = 0.63$ (B)

Because (B) depends on (A), $p(\text{B and A}) = p(\text{A}) * p(\text{B}) = p(\text{A}) * p(\text{B})$

Log-Likelihood, and Negative Log-Likelihood

- The probability of ground truth token can be logged(logarithms), which maps multiplication to sum

 Log-Likelihood(LL)

- The LL can be inversed, so it maps the intuition that larger value means unexpectedness

 Negative Log-Likelihood(NLL)

- * Negative Log-likelihood is also called surprisal

$$\text{Surprisal } I(w_i | w_{i-n} \dots w_{i-1}) = -\log (p(w_i | w_{i-n} \dots w_{i-1}))$$

Perplexity(PPL)

- Def: Exponentiated log-likelihood of sequence of tokens

Input: M: Large Language Model)
X: Text, parsed as sequence of tokens $\{x_1, x_2, x_3, \dots, x_n\}$

$$ppl(M, X) = \exp \left\{ -\frac{1}{t} \sum_i^t \log(p_M(x_i | x_{<i})) \right\}$$

$p_M(x_i | x_{<i})$: M-predicted likelihood of each token given previous tokens

- Predictability of a LM over a sequence of token(e.g. a piece of text)
- Lower PPL, higher predictability easier the text for the model

PPL: Example

$$ppl(M, X) = \exp \left\{ -\frac{1}{t} \sum_i^t \log(p_M(x_i | x_{<i})) \right\}$$

I get two things: an

apple	and	a	pear.
-------	-----	---	-------

Token	Token	Token	Token	Prob
apple	and	a	pear	0.56
orange	,a	an	orange	0.3
pear	pear	.	apple	0.01
...	...	...	...	...

$p_M(\text{apple} | \text{I get two things: an}) = 0.63$

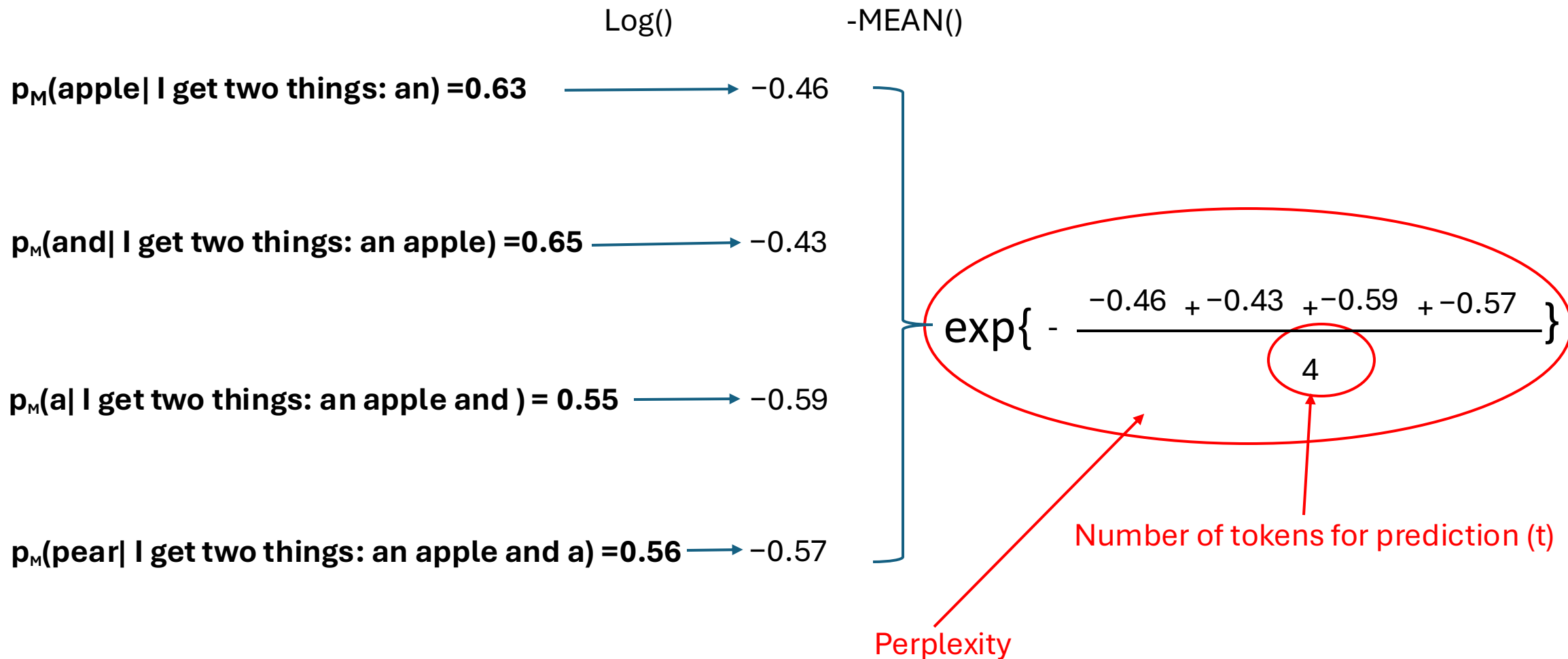
$p_M(\text{and} | \text{I get two things: an apple}) = 0.65$

$p_M(\text{a} | \text{I get two things: an apple and}) = 0.55$

$p_M(\text{pear} | \text{I get two things: an apple and a}) = 0.56$

PPL: Example

$$ppl(M, X) = \exp \left\{ -\frac{1}{t} \sum_i^t \log(p_M(x_i | x_{<i})) \right\}$$



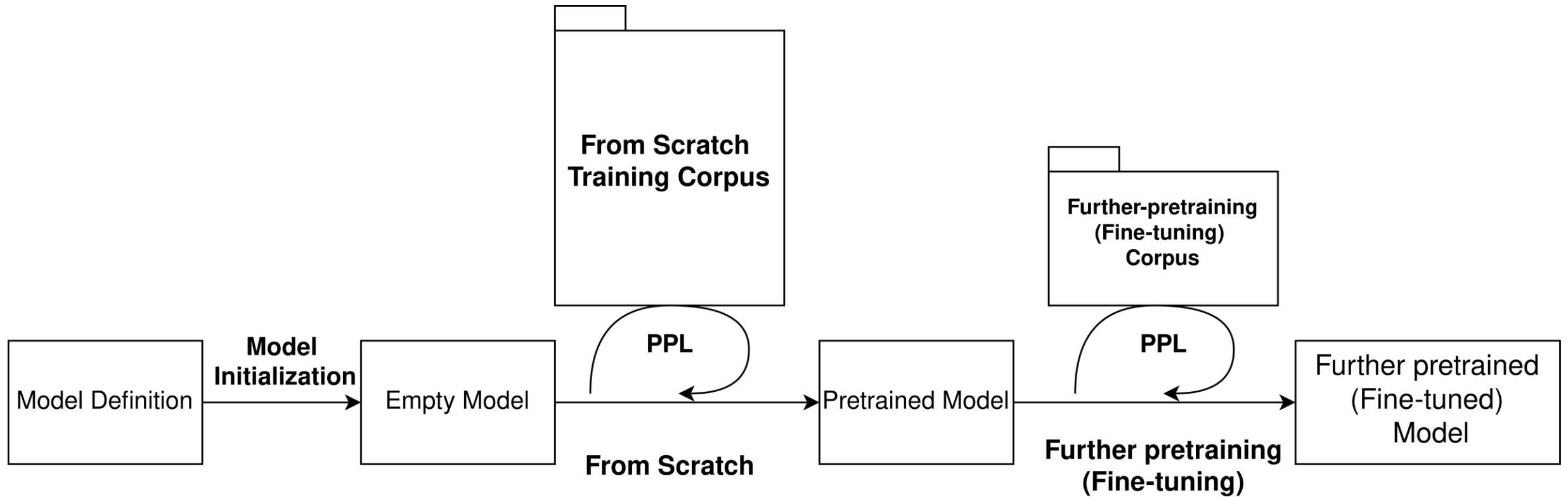
Model Inferring & Training

- Model inferring is the process that model performs assigned task
 - Input: tokenized text
 - Output: probability of token(s)
- Model training is a way of updating the parameters of a model ...
 - Input: tokenized texts(training corpus)
 - Output: model parameters(weight & biases)
 - Optimization: Minimize PPL of LLM over input(training) text

Types of Model Training

- Training from scratch
 - no imported initial weights
- Fine-tuning
 - with imported weights from other, readily-trained model
 - Task Maintained: Further pretraining
 - Taks Changed: Fine-tuning to another task(e.g. language model as classifier)

Example of Training Pipeline



Note for Session 1B

- LLM predicts probability of tokens based on their context.
- Probability of ground truth token(s) show how a sequence is "expected" by LLM.
 - Text level: PPL
 - Token level: NLL
 - Higher value = less expected = more surprising
- Training/Fine-tuning updates LLM parameters
 - According to input texts.
 - Via minimizing PPL.
 - "Make model learn/familiarize the input texts"

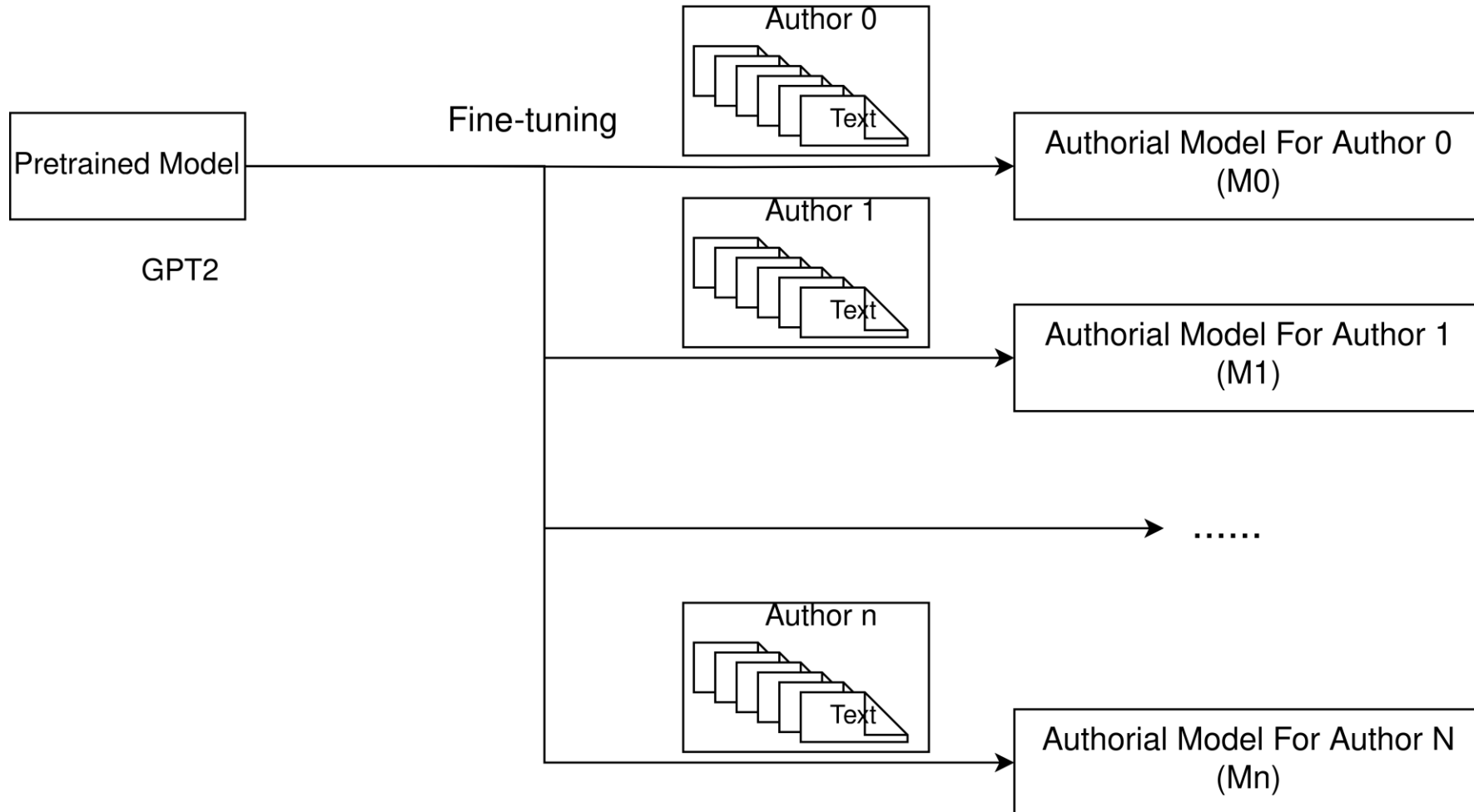
Large Language Model For Authorship Attribution

ALMs: Authorial Language Models

- Authorship attribution method based on PPL of authorial fine-tuned LLM
- Steps
 - Automated Authorship Attribution:
 - Fine-tuning of LMs for each candidate author
 - Calculate PPLs of the question text on each authorial LM
 - Predict author by lowest PPL
 - Computer-assisted Authorship Attribution
 - Token-level text annotation

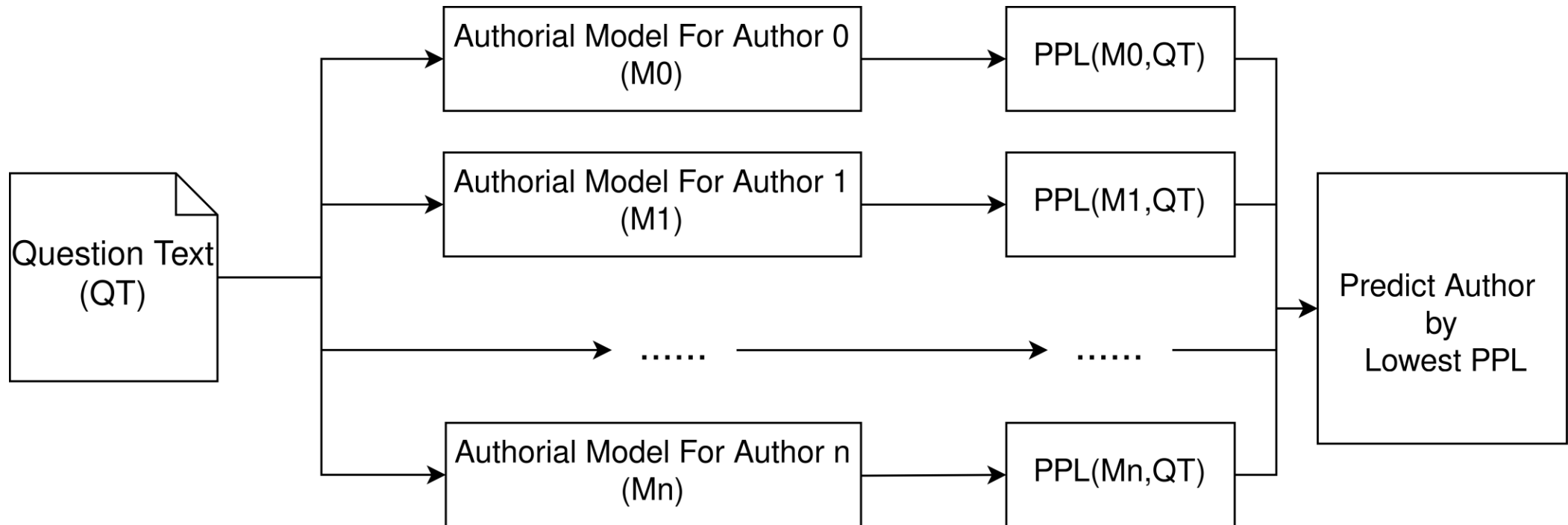
ALMs:

Steps: Fine-tuning of LMs For Each Author



ALMs:

Steps: Calculate ALMs PPLs & Predict Author



Data

Table 1. Facts on datasets.

Dataset	A	T	TK	T/A	MTL	Register
CCAT50	50	5k	2.5M	100	506	News
Blogs50	50	66k	8.1M	1.3k	122	Blogs
Guardian	13	444	467k	34	1052	News
IMDB62	62	62k	21.6M	1k	349	Film Comments

A: author count; T: text count in total; TK: token count in total; T/A: text count per author; MTL: mean text length (i.e. token count per text); Register: brief description of register of texts in the dataset.

Result

Method	Variable	Blogs50	CCAT50	Guardian	IMDB62	Mean
ALMs	Acc.	83.6	74.9	94.5	99.5	88.1
	SE	0.1	1.2	0.5	0.1	0.5
Ngram	Acc.	72.3	76.7	100	98.8	87.0
	SE	0.4	1.5	0.0	0.0	0.5
BERT	Acc.	75	65.7	84.2	98.8	80.9
	SE	0.7	1.3	0.7	0.1	0.7
PPM	Acc.	72.2	69.4	86.3	95.9	81.0
	SE	0.6	1.4	0.7	1.0	0.9
pALM	Acc.	69.3	63.4	66.6	90.4	72.4
	SE	0.3	1.5	0.6	0.9	0.8
Random Baseline	Acc.	2	2	7.7	1.6	3.3

Token-Level Text Annotation

- NLL is measured with token-level granularity
- Higher NLL, less predictable the token is by an ALM
- Color-coded NLL can be an effective way in labelling suspicious tokens that are not expected by an ALM

Token-Level Analysis: A Closer Look

Question Text: Jacob Abbott's *Rollo Books Series*

The children followed it along down the bro ok , and began to p elt it with stones , and soon got into a high fro lic . But as they were very careful not to hit one another with the stones , nor to speak harshly or cross , they enjoyed it very much . When at last the steam - boat was fairly pel ted to pieces , and the black ened fragments of the bir ch bark were scattered over the water , and floating away down the stream , they began to think of ro asting their corn and potatoes , which they did very successfully over the remains of the fires .

Token-Level Analysis: A Closer Look

Question Text: Jacob Abbott's *Rollo Books Series*

PPL(QT, Mrs. Alexander) Annie French Hector

The children followed it along down the brook , and began to pelt it with stones , and soon got into a high frolic . But as they were very careful not to hit one another with the stones , nor to speak harshly or cross , they enjoyed it very much . When at last the steam - boat was fairly pelted to pieces , and the blackened fragments of the birch bark were scattered over the water , and floating away down the stream , they began to think of roasting their corn and potatoes , which they did very successfully over the remains of the fires .

Infinitive "to"

ALMs-NLL	to	to	to	to	to
Func. Freq.	to	to	to	to	to

Token-Level Analysis: A Closer Look

Question Text: Jacob Abbott's *Rollo Books Series*

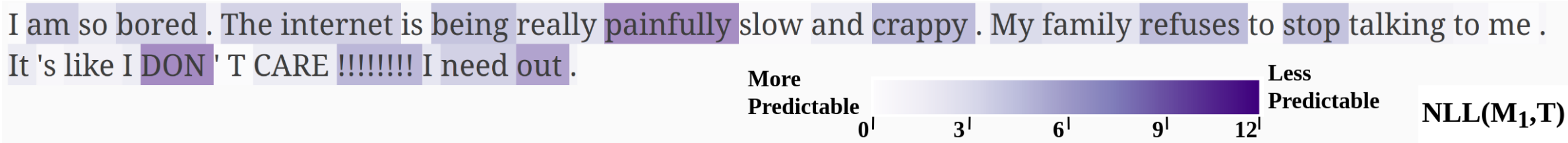
PPL(QT, Mrs. Alexander) Annie French Hector

The children followed it along down the brook , and began to pelt it with stones , and soon got into a high frolic . But as they were very careful not to hit one another with the stones , nor to speak harshly or cross , they enjoyed it very much . When at last the steam - boat was fairly pelted to pieces , and the blackened fragments of the birch bark were scattered over the water , and floating away down the stream , they began to think of roasting their corn and potatoes , which they did very successfully over the remains of the fires .

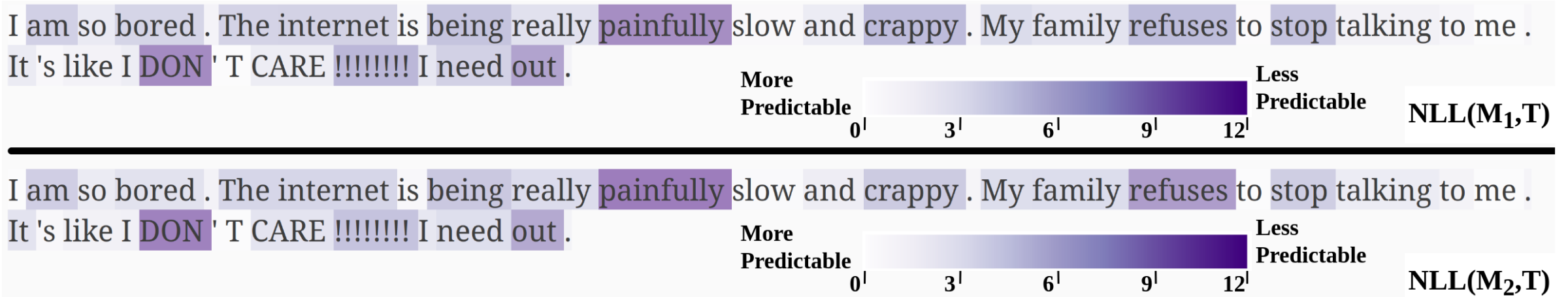
Prep"of"

ALMs-NLL	of	of
Func. Freq.	of	of

Token-level text annotation(NLL)



Token-level text annotation(NLL)



Comparative Negative Log-Likelihood(CNLL)

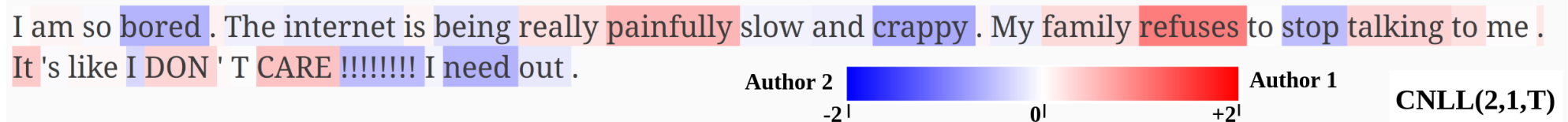
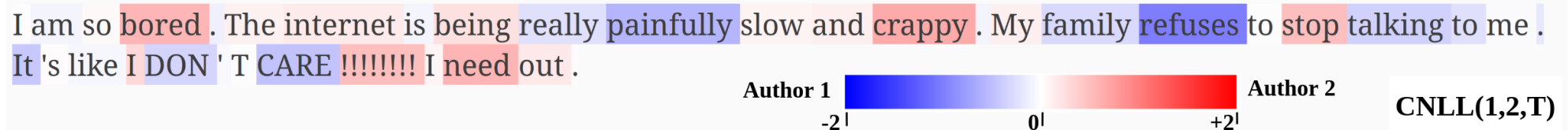
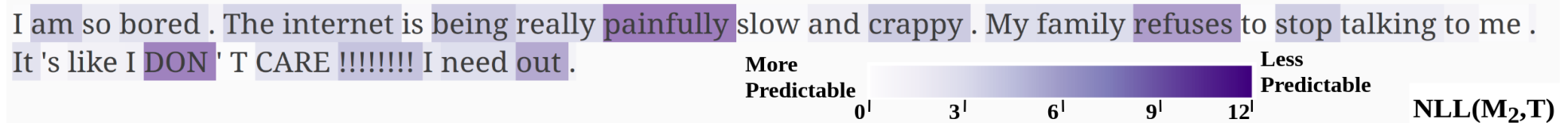
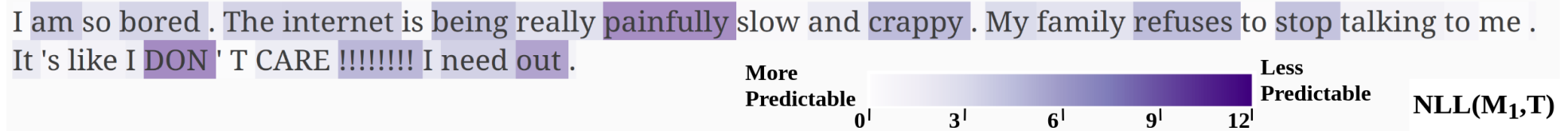
- Comparative Negative Log-likelihood
- Pairwise(comparison against another candidate author)

$$\text{CNLL}(a, b, i) = NLL_{m_a, i} - NLL_{m_b, i}$$

- One versus rest(comparison against a pool of candidate authors)

$$\text{CNLL}(a, C, i) = NLL_{m_a, i} - \frac{1}{n-1} \sum_{b \in (C \setminus \{a\})} NLL_{m_b, i}$$

CNLL for Pairwise Comparison

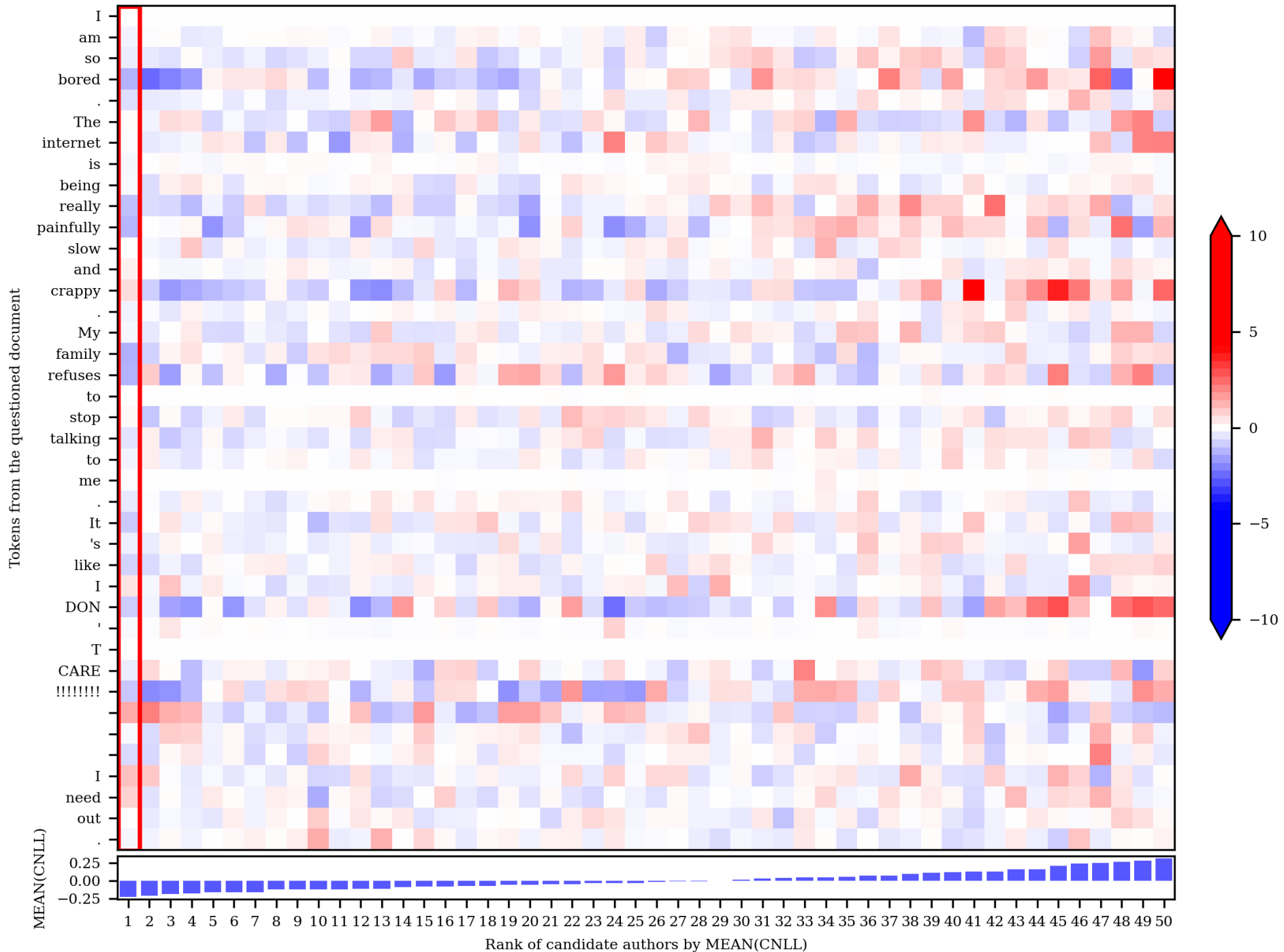


CNLL One Versus Rest Text Annotation

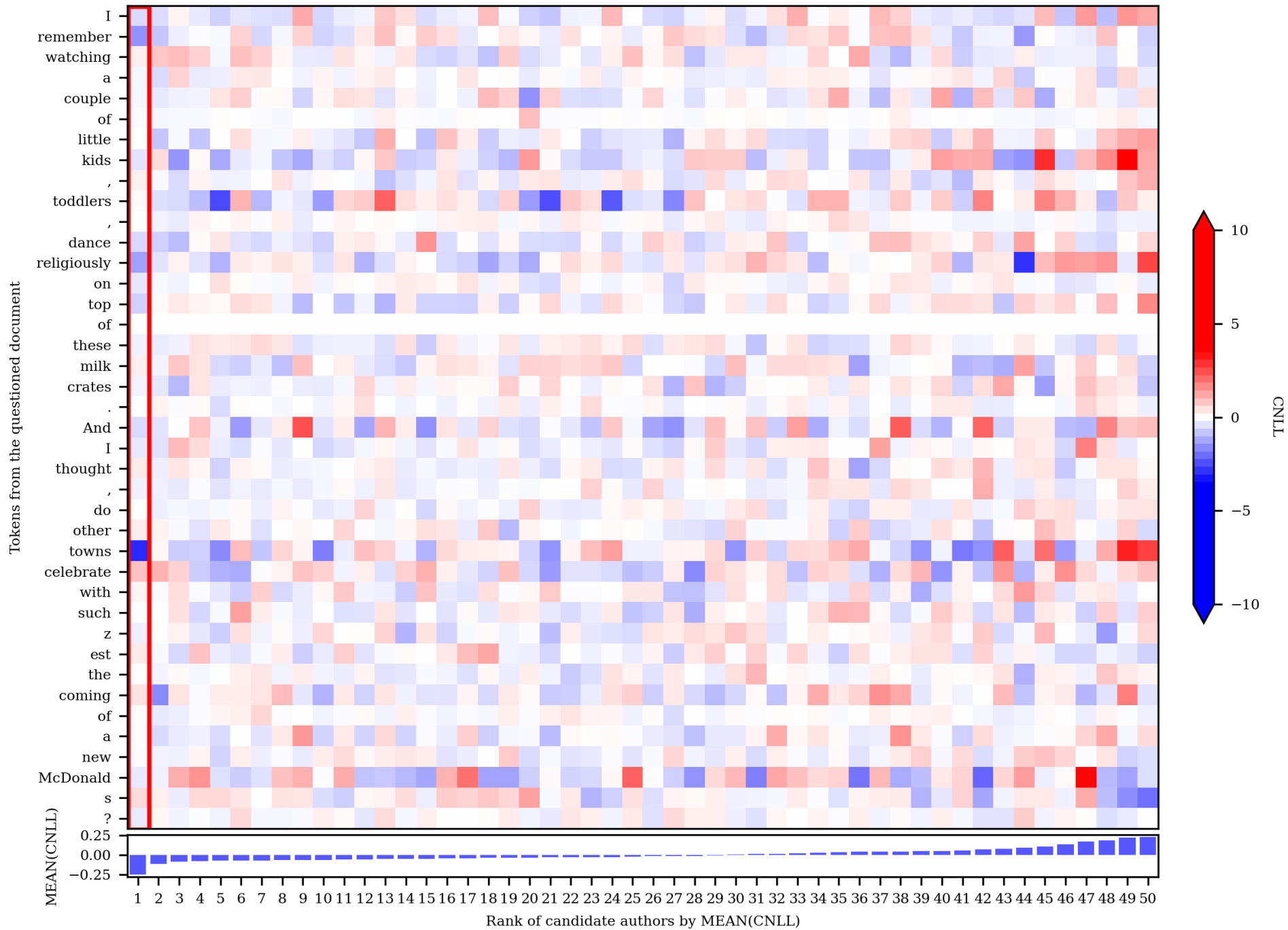
I am so bored . The internet is being really painfully slow and crappy . My family refuses to stop talking to me . It 's like
I DON ' T CARE !!!!!!!!! I need out .

Author 1 supported -2 0 +2 Author 1 refuted CNLL(1,C,T)





CNLL
One Versus Rest
In
Heatmap
Example 1



CNLL
One Versus Rest
In
Heatmap
Example 2

Conclusion

- ALMs is an effective method in authorship attribution that ...
 - Utilize fine-tuned large language models
 - Achieve high performance score at corpora with (even)...
 - Large author #
 - Low question text length
 - Feasible for token-level explanation & interpretation
- ALMs took the advantages from both stylistics and stylometry

Q & A

Thank you.



UNIVERSITY OF
BIRMINGHAM

Computational Authorship Analysis Part II

Authorship Attribution via Large Language Models

Weihang Huang

Department of Linguistics and Communication

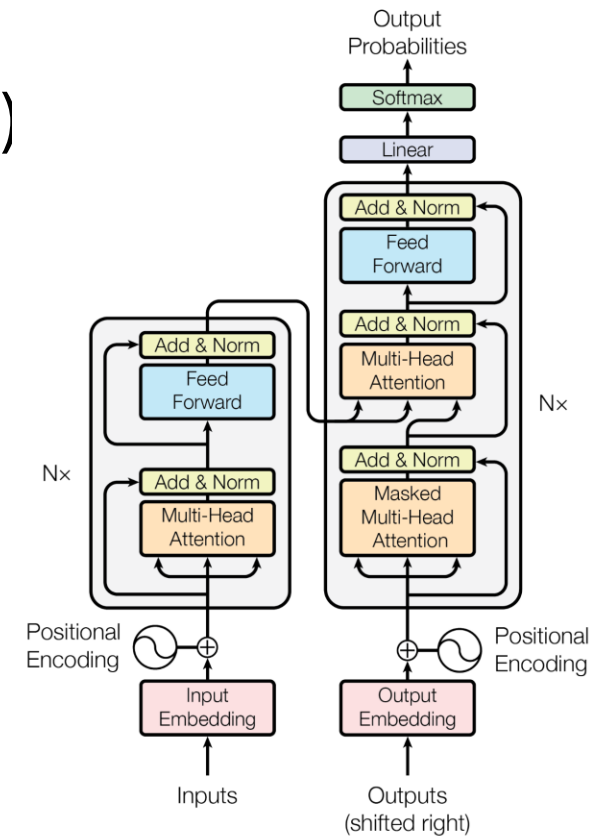
University of Birmingham

QA1:Language Model: Category

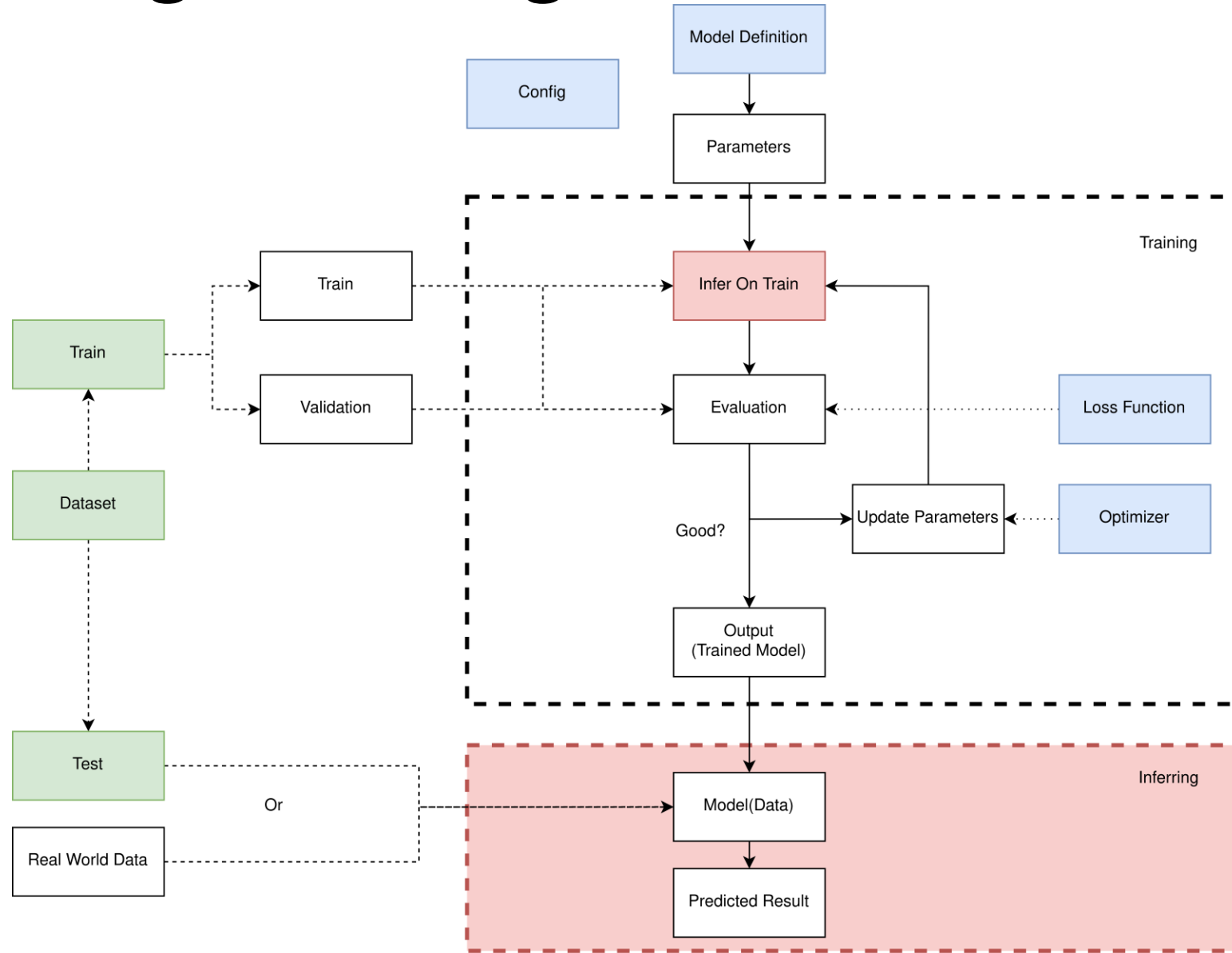
- Category
 - Context
 - Range
 - Causal Language Model: predict next words(e.g. GPT-2/3/4)
 - Masked Language Model: predict the <MASK> in text(e.g. BERT, RoBERTa)
 - ...
 - Context split
 - One(context only)
 - Two(context-instruct/question split for Q&A bot)
 - ...
 - Application
 - Language Generator/Text Generating Machine(e.g. raw GPT-2/3/4, Liama-2)
 - Question Answering Bot(e.g. ChatGPT, Claude, OpenChat)
 - ...and more

QA2:Language Model: How Large Is Large?

- With billions of parameters
- Typically, after the invention of transformers(2017)



QA3: Training / Inferring Framework



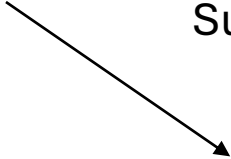
QA4: ALMs vs. Stylistic vs. Stylometric

	Stylistic	Stylometric	ALMs
Implementation	Manual (Close Reading)	Computational (Distant Reading)	Computational (Distant Reading)
Feature Selection	Discovered (Bottom up)	Pre-defined (Top down)	Discovered (Bottom up)
Feature Type	Occurrence of Tokens	Frequency of Types	Occurrence of Tokens
Quantification	Minimal	Central	Central
Contextualisation	Central	Minimal	Central
Explanation	Moderate	Minimal	Facilitated

QA5: CE Further investigation

$$H(p, q) = -\mathbb{E}_p[\log q]$$

Assumption: Discrete Probability Distribution
Support uniquely = $\mathcal{X}$



$$H(p, q) = -\sum_{x \in \mathcal{X}} p(x) \log q(x)$$

Assumption: Monte Carlo Estimation Enabled



$$H(T, q) = -\sum_{i=1}^N \frac{1}{N} \log_2 q(x_i)$$

QA5A: Quick Flash on Monte Carlo at Practice

- I have two things an apple and a pear.
- Real P: PT@Real

	...	a	abandon	...	an	...	apple	...
I	...	0.02581	0.07568	...	0.06320	...	0.03982	...
get	...	0.00039	0.00399	...	0.09626	...	0.04648	...
two	...	0.00185	0.00996	...	0.01128	...	0.24887	...
things	...	0.00390	0.00625	...	0.83658	...	0.00310	...
an	...	0.24771	0.01989	...	0.42341	...	0.21486	...
apple	...	0.00042	0.00016	...	0.08132	...	0.38014	...
and	...	0.00451	0.00288	...	0.16216	...	0.01336	...
a	...	0.23962	0.00017	...	0.11122	...	0.13319	...
pear	...	0.00025	0.00042	...	0.00285	...	0.12497	...

QA5A: Quick Flash on Monte Carlo at Practice

- I have two things an apple and a pear.
- Estimated P: One hot encoded vectors as distribution

	...		a	abandon	...	an	...	apple	...
I	...		0	0	...	0	...	0	...
get	...		0	0	...	0	...	0	...
two	...		0	0	...	0	...	0	...
things	...		0	0	...	0	...	0	...
an	...		0	0	...	1	...	0	...
apple	...		0	0	...	0	...	1	...
and	...		0	0	...	0	...	0	...
a	...		1	0	...	0	...	0	...
pear	...		0	0	...	0	...	0	...

QA5: CE Further investigation

- $P(x) = [0, 0, 0, \dots, 1, \dots, 0]$
- $Q(x) = [p(\text{LM lexi1} | \text{context}), p(\text{LM lexi2} | \text{context}), \dots]$
- $P(x) * Q(x) = p(\text{LM lexi ground truth} | \text{context}) = p(x_i | x_{<i}) = \text{likelihood}$

Output of LLM: Logits Matrix

- Logits matrix is the raw output of large language model.
 - Row label: Tokens in input sequence
 - Column label: producible tokens, as defined by LLM
 - Logits: Ratio of probability of each producible token for that token in input sequence

Logits Matrix: A Closer Look

LLM producible tokens

Tokens in input sequence

Logits

	!	"	...	Today	absence	...	<EOS>
I	-39.31	-39.01	...	-43.79	-44.71	...	-39.51
get	-106.20	-105.15	...	-114.06	-112.95	...	-107.85
two	-104.06	-102.34	...	-112.67	-111.05	...	-106.08
things	-113.86	-113.41	...	-123.30	-126.89	...	-116.27
an	-89.49	-87.19	...	-91.35	-88.28	...	-89.44
apple	-105.93	-105.21	...	-114.36	-116.90	...	-108.25
and	-116.20	-115.36	...	-124.98	-120.72	...	-116.25
a	-116.82	-114.54	...	-124.31	-120.90	...	-116.08
pear	-102.46	-102.42	...	-111.51	-114.72	...	-106.69

How to Read Logits

	!	"	...	Today	absence	...	<EOS>
I	-39.31	-39.01	...	-43.79	-44.71	...	-39.51
get	-106.20	-105.15	...	-114.06	-112.95	...	-107.85
two	-104.06	-102.34	...	-112.67	-111.05	...	-106.08
things	-113.86	-113.41	...	-123.30	-126.89	...	-116.27
an	-89.49	-87.19	...	-91.35	-88.28	...	-89.44
apple	-105.93	-105.21	...	-114.36	-116.90	...	-108.25
and	-116.20	-115.36	...	-124.98	-120.72	...	-116.25
a	-116.82	-114.54	...	-124.31	-120.90	...	-116.08
pear	-102.46	-102.42	...	-111.51	-114.72	...	-106.69

How to Read Logits

	!	"	...	Today	absence	...	<EOS>
apple	-105.93	-105.21	...	-114.36	-116.90	...	-108.25

The Ratio of likelihood of token
!, ", Today, absence and <EOS>
is
-105.93 : -105.21 : -114.36 : -116.90 : -108.25

Softmax: From Logits To Probability

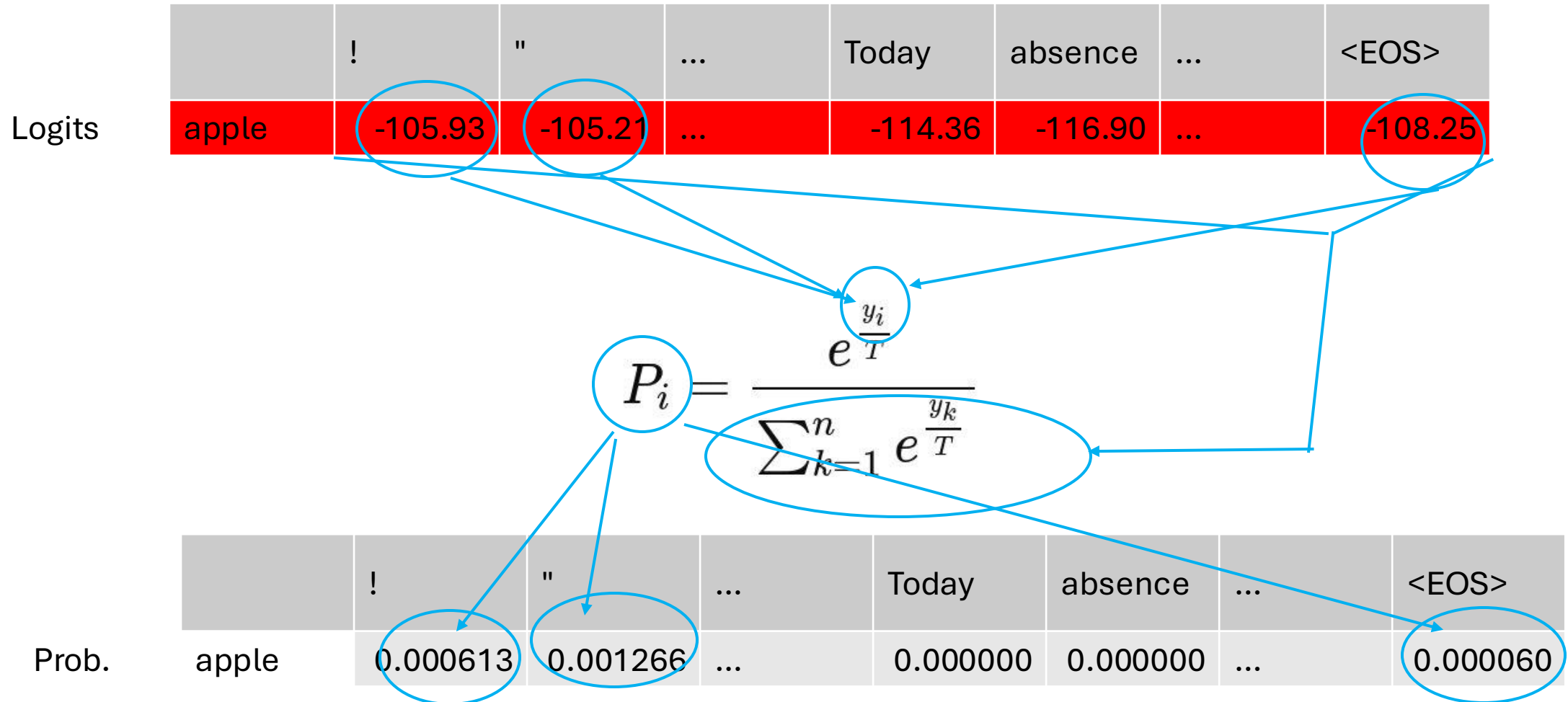
- Task: map a row of Logits into probability distribution of 0 to 1
- Formula: for each producible token **i** in a row of **k** Logits

The diagram illustrates the Softmax formula with the following components and annotations:

- Estimated Probability**: A red arrow points to the variable P_i in the numerator.
- Logit at producible token i**: A red arrow points to the variable y_i in the numerator.
- Temperature**: A red arrow points to the variable T in the denominator.
- Sum of exponentiated logits of all producible token**: A red arrow points to the summation term $\sum_{k=1}^n e^{\frac{y_k}{T}}$ in the denominator.
- Base of natural logarithm ~2.718**: A blue box at the bottom contains this text, with blue arrows pointing to the base e in both the numerator and denominator.

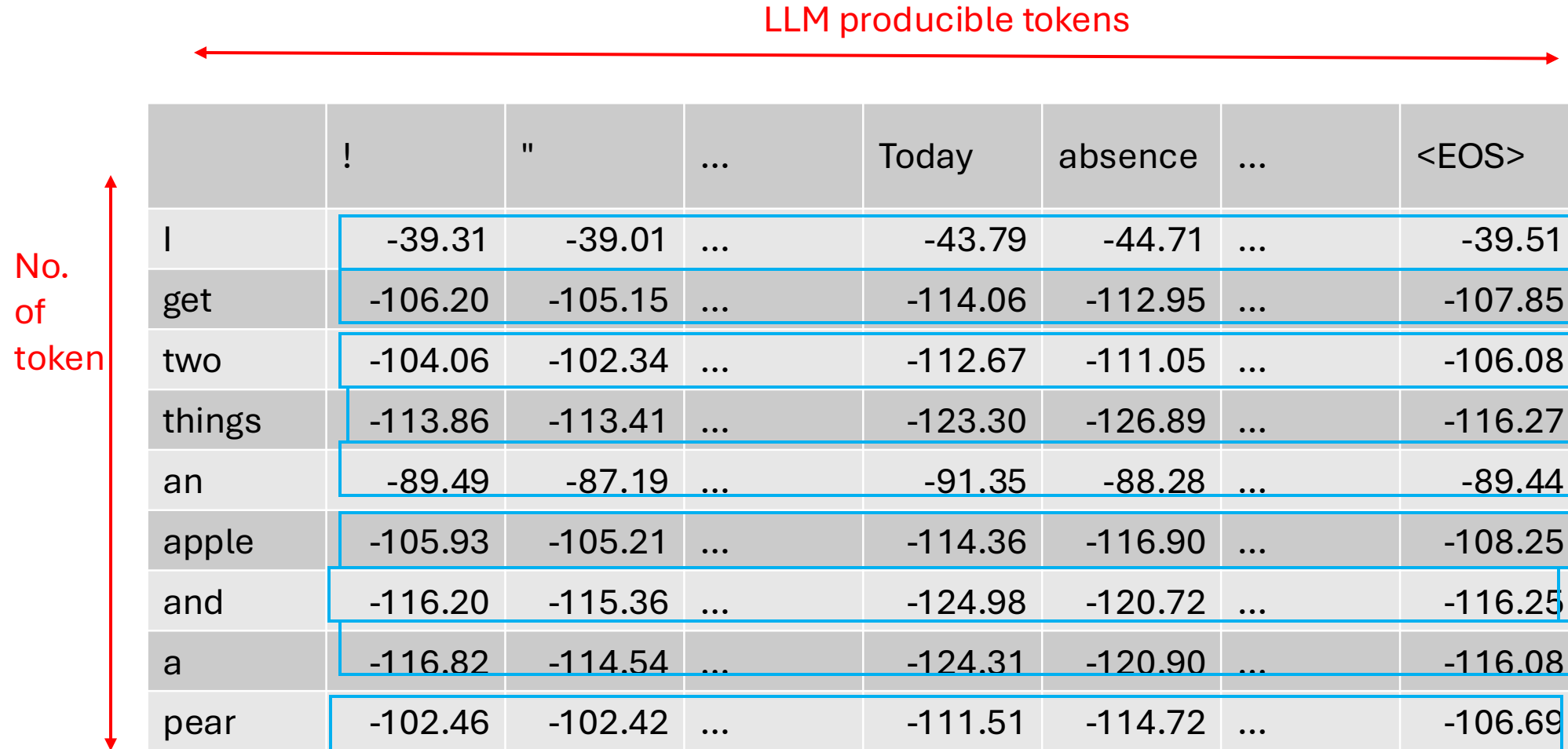
$$P_i = \frac{e^{\frac{y_i}{T}}}{\sum_{k=1}^n e^{\frac{y_k}{T}}}$$

Softmax: A Closer Look



SoftMax: From Logits To Probability

LLM producible tokens



	!	"	...	Today	absence	...	<EOS>
I	-39.31	-39.01	...	-43.79	-44.71	...	-39.51
get	-106.20	-105.15	...	-114.06	-112.95	...	-107.85
two	-104.06	-102.34	...	-112.67	-111.05	...	-106.08
things	-113.86	-113.41	...	-123.30	-126.89	...	-116.27
an	-89.49	-87.19	...	-91.35	-88.28	...	-89.44
apple	-105.93	-105.21	...	-114.36	-116.90	...	-108.25
and	-116.20	-115.36	...	-124.98	-120.72	...	-116.25
a	-116.82	-114.54	...	-124.31	-120.90	...	-116.08
pear	-102.46	-102.42	...	-111.51	-114.72	...	-106.69

Data Facts

author_tag	textcount	tokencount	mtl
satoshi	503	59,989	119.26
adam3us	404	132,118	327.02
Peter Todd	1,235	228,810	185.27
Hal	299	34,794	116.37
Gavin Andresen	2,168	284,419	131.19